NATIONAL UNIVERSITY OF SINGAPORE

CS2030S– Programming Methodology II

FINAL

November 2022 TIME ALLOWED: 120 MINS

INSTRUCTIONS TO CANDIDATES

1. This assessment paper contains **THIRTY (30)** questions and comprises **TWENTY (22)** printed pages.
2. Write all your answers in the answer sheet provided.
3. The total mark for this assessment is 90.
4. This is an **OPEN BOOK** assessment (*hardcopies only*).
5. All questions in this assessment paper uses Java 17.
6. Answer **ALL** questions.
7. Question types:
    - **MCQ:** Select the most appropriate answer. If multiple answers are equally appropriate, pick ONE and write the chosen answer (*i.e., the letters*) in the answer box.
    - **MRQ:** Select ALL correct answers. Write ALL the chosen answers (*i.e., the letters*) in the answer box. No partial mark given.
    - **ESSAY:** Write your answer in the box provided.
    - **TRUE/FALSE:** Write True or False as well as any explanations needed.

Section I: Basic (10 points)

This section tests the basic of Java. Furthermore, by *necessary*, we meant that without such requirements/conditions, the property is no longer satisfied. If the requirements/conditions are *optional* (*i.e.,* it can be omitted and the property is still satisfied), they are not considered necessary.

1. [**MRQ**] *(2 points)* Select ALL the requirements below that are *necessary* to make an interface a functional interface.

   A. The interface must have a generic type (*e.g.,* `interface I<T> {}`)

   B. The interface must be annotated with `@FunctionalInterface`

   C. The interface cannot extend another interface

   D. The interface must have only a single abstract method

   E. *None of the other*

2. [**MRQ**] *(2 points)* Consider the static method `fn` below:

   ```
   class Main {
     static <T> T fn(T... args) { ... }
   }
   ```

   Select ALL statements that are true about `fn`.

   A. The arguments `args` is a `List` of `T` (*i.e.,* `List<T>`) inside the method `fn`

   B. `fn` may be called with 0 argument

   C. `fn` must be called with at least 1 argument

   D. We may pass a `List` into `fn` (*e.g.,* `Main.fn(List.of(1, 2, 3));`)

   E. *None of the other*

3. [**MRQ**] *(2 points)* Select ALL the *necessary* conditions to make a class *immutable*.

   A. All fields must have the `final` modifier

   B. All constructors must have `private` modifier

   C. All methods must have the `final` modifier

   D. All methods must return a new instance

   E. *None of the other*

4. [**MRQ**] *(2 points)* Select ALL the true statement about our `Memo<T>` class.

   A. It only does the computation at the last minute

   B. It will perform the computation to obtain the value while other computations are running in a different thread

   C. It never repeat the computation to obtain the value

   D. It may prevent errors from happening if the content is never computed

   E. *None of the other*

5. [**MRQ**] *(2 points)* Select ALL the true statement about *local* class.

   A. It must not have a name

   B. It must be defined within another class

   C. It must be defined within a (*non-static*) method

   D. It may access the method parameter but may not modify it

   E. *None of the other*

Section II: Nested Classes (12 points)

Consider the following class and interface:

```
class Int {
  int val;
  Int(int val) {
    this.val = val;
  }
}
interface Eq {
  boolean check(Object x, Object y);
}
```

6. [**MCQ**] *(3 points)* Consider the following code:

```
class Outer {
  int x = 0;
  Eq foo() {
    return new Eq() {
      public boolean check(Object a, Object b) {
        if (x == 0) {
          return a.equals(b);
        } else {
          return !a.equals(b);
        }
      }
    };
  }
  public static void main(String[] args) {
    Outer out = new Outer();
    Eq eq = out.foo();
    out.x = 1;
    boolean res = eq.check(1,1);
  }
}
```

Which of the following statement is true about the code above?

A. The code does not compile because `x` is not *effectively final*

B. The code does not compile because `Eq` is an *interface* and cannot be instantiated

C. The code compiles and the value of `res` is `true` at the end

D. The code compiles and the value of `res` is `false` at the end

E. *None of the other*

7. [**MCQ**] *(3 points)* Consider the following code:

```
class Outer {
  Eq foo(int x) {
    return new Eq() {
      public boolean check(Object a, Object b) {
        if (x == 0) {
          return a.equals(b);
        } else {
          return !a.equals(b);
        }
      }
    };
  }
  public static void main(String[] args) {
    Outer out = new Outer();
    int x = 0;
    Eq eq = out.foo(x);
    x = 1;
    boolean res = eq.check(1,1);
  }
}
```

Which of the following statement is true about the code above?

A. The code does not compile because `x` is not *effectively final*

B. The code does not compile because `Eq` is an *interface* and cannot be instantiated

C. The code compiles and the value of `res` is `true` at the end

D. The code compiles and the value of `res` is `false` at the end

E. *None of the other*

8. [**MCQ**] *(3 points)* Consider the following code:

```
class Outer {
  Eq foo(int x) {
    x = 1;
    return new Eq() {
      public boolean check(Object a, Object b) {
        if (x == 0) {
          return a.equals(b);
        } else {
          return !a.equals(b);
        }
      }
    };
  }
  public static void main(String[] args) {
    Outer out = new Outer();
    int x = 0;
    Eq eq = out.foo(x);
    boolean res = eq.check(1,1);
  }
}
```

Which of the following statement is true about the code above?

A. The code does not compile because `x` is not *effectively final*

B. The code does not compile because `Eq` is an *interface* and cannot be instantiated

C. The code compiles and the value of `res` is `true` at the end

D. The code compiles and the value of `res` is `false` at the end

E. *None of the other*

9. [**MCQ**] *(3 points)* Consider the following code:

```
class Outer {
  Eq foo(Int x) {
    x.val = 1;
    return new Eq() {
      public boolean check(Object a, Object b) {
        if (x.val == 0) {
          return a.equals(b);
        } else {
          return !a.equals(b);
        }
      }
    };
  }
  public static void main(String[] args) {
    Outer out = new Outer();
    Int x = new Int(0);
    Eq eq = out.foo(x);
    boolean res = eq.check(1,1);
  }
}
```

Which of the following statement is true about the code above?

A. The code does not compile because `x` is not *effectively final*

B. The code does not compile because `Eq` is an *interface* and cannot be instantiated

C. The code compiles and the value of `res` is `true` at the end

D. The code compiles and the value of `res` is `false` at the end

E. *None of the other*

Section III: PECS (12 points)

Recap our functional interface in `cs2030s.fp` package:

```
interface Immutator<R,P> {
  R invoke(P p);
}
```

Further consider the subtyping relationship `C <: B <: A`. In this question, we will vary the type signature of the method `apply` below by changing `R` and `P` into wild cards.

```
class Application {
  static <R,P> R apply(Immutator<R, P> fn) {
    return null;
  }
}
```

We will be using the following variables:

```
Immutator<A,A> f1 = arg -> new A();
Immutator<A,C> f2 = arg -> new A();
Immutator<B,B> f3 = arg -> new B();
Immutator<C,A> f4 = arg -> new C();
Immutator<C,C> f5 = arg -> new C();
```

10. [**MRQ**] *(4 points)* Consider the following method signature:

    ```
    static <R,P> R apply(
      Immutator<? super R, ? extends P> fn
    );
    ```

    Select ALL code fragments that does not cause *compile-error*.

    A. `Application.<B,B>apply(f1);`

    B. `Application.<B,B>apply(f2);`

    C. `Application.<B,B>apply(f3);`

    D. `Application.<B,B>apply(f4);`

    E. `Application.<B,B>apply(f5);`

11. [**MRQ**] *(4 points)* Consider the following method signature:

```
static <R,P> R apply(
  Immutator<? extends R, ? super P> fn
);
```

Select ALL code fragments that does not cause *compile-error*.

A. `Application.<B,B>apply(f1);`

B. `Application.<B,B>apply(f2);`

C. `Application.<B,B>apply(f3);`

D. `Application.<B,B>apply(f4);`

E. `Application.<B,B>apply(f5);`

12. [**MRQ**] *(4 points)* Consider the following method signature:

```
static <R,P> R apply(
  Immutator<?, ?> fn
);
```

Select ALL code fragments that does not cause *compile-error*.

A. `Application.<B,B>apply(f1);`

B. `Application.<B,B>apply(f2);`

C. `Application.<B,B>apply(f3);`

D. `Application.<B,B>apply(f4);`

E. `Application.<B,B>apply(f5);`

Section IV: Functional Interface (16 points)

Recap our functional interface in `cs2030s.fp` package:

```
interface Immutator<R,P> {
  R invoke(P p);
}
```

Further consider the following minimal definition of a `Box<T>`:

```
class Box<T> {
  private final T val;
  private Box(T val) {
    this.val = val;
  }
  public static <T> Box<T> of(T val) {
    return new Box<>(val);
  }
}
```

13. [**MCQ**] *(4 points)* We want to add a new interface which extends `Immutator`, to capture an operation that takes in an input of generic type `T` and returns an instance of the same generic type `T`. Let's call this interface an `Operation<T>`. Which of the following interface definition below captures the idea above such that the definition is *as flexible as possible* while still compiles?

    A.
    ```
    interface Operation<T> {
      T invoke(T t);
    }
    ```

    B.
    ```
    interface Operation<T> extends Immutator<T, T> {
    }
    ```

    C.
    ```
    interface Operation<T> extends Immutator<? extends T, ? super T> {
    }
    ```

    D.
    ```
    interface Operation<T> extends Immutator<? super T, ? extends T> {
    }
    ```

    E. *None of the other*

14. [**MRQ**] *(4 points)* Say we have already created the interface `Operation<T>` correctly. We want to use this interface in the generic `Box<T>` in the method `Box::ops(Operation fn)` that behaves in a similar way as `map` method. We want this method to be *as flexible as possible* while still compiles. Which of the following definition is the *most flexible* definition of `Box::ops(Operation fn)`? If there are multiple *most flexible* definition, select ALL of them.

    Choose *"None of the other"* only if there are no other correct answer.

    A. `public Box<T> ops(Operation<? extends T> fn) { ... }`

    B. `public Box<T> ops(Operation<? super T> fn) { ... }`

    C. `public Box<T> ops(Operation<T> fn) { ... }`

    D. `public Box<T> ops(Operation<?> fn) { ... }`

    E. *None of the other*

15. [**MCQ**] *(4 points)* Instead of `Operation<T>`, we now want to add more parameter to `Immutator<R,P>` and create *an abstract class* for this. Let's call this abstract class `BiImmutator<R,P,Q>`. We define this as follows:

```
abstract class BiImmutator<R,P,Q> implements
      Immutator<Immutator<R, P>, Q> {
  public R invoke(P p, Q q) {
    // implementation hidden
  }
}
```

    Which of the following implementation can be used to replace the hidden implementation above?

    A.
```
public R invoke(P p, Q q) {
  return q.invoke(this.invoke(p));
}
```

    B.
```
public R invoke(P p, Q q) {
  return p.invoke(this.invoke(q));
}
```

C.
```
public R invoke(P p, Q q) {
  return this.invoke(p).invoke(q);
}
```

D.
```
public R invoke(P p, Q q) {
  return p.invoke(q.invoke(this));
}
```

E. *None of the other*

16. [**MCQ**] *(4 points)* Next, we want to create a non-generic class `Subtractor` such that it has a method called `invoke` that takes in two arguments `x` and `y` and returns the result of subtracting `y` from `x` (*i.e.,* `new Subtractor().invoke(x, y)` is equivalent to `x - y`).

We assume that the *correct* version of `public R invoke(P p, Q q)` has been given (*i.e.,* the previous question has been answered correctly). We use the following class definition:

```
class Subtractor extends BiImmutator<Integer, Integer, Integer> {
  @Override
  public /* return type */ invoke(/* parameters */) {
    // implementation hidden
  }
}
```

Which of the following implementation can be used to replace the hidden implementation above with the correct return type and parameters such that the code compiles?

If there are multiple possible answers, choose any of the possible answer.

A.
```
class Subtractor extends BiImmutator<Integer,Integer,Integer> {
  @Override
  public Immutator<Integer,Integer> invoke(Integer x) {
    return y -> y - x;
  }
}
```

B.
```
class Subtractor extends BiImmutator<Integer,Integer,Integer> {
  @Override
  public Immutator<Integer,Integer> invoke(Integer x) {
    return y -> x - y;
  }
}
```

C.
```
class Subtractor extends BiImmutator<Integer,Integer,Integer> {
  @Override
  public Integer invoke(Integer x, Integer y) {
    return x - y;
  }
}
```

D.
```
class Subtractor extends BiImmutator<Integer,Integer,Integer> {
  @Override
  public Integer invoke(Integer x, Integer y) {
    return y - x;
  }
}
```

E. *None of the other*

Section V: CompletableFuture (9 points)

We will use the following methods in `CompletableFuture<T>` for this section. Additionally, we have removed some wild cards to simplify the method signature.

- `CompletableFuture<Void> thenRun(Runnable action)`: Returns a new `CompletionStage` that executes the given `action`.
- `CompletableFuture<Void> runAfterBoth(CompletionStage<?> other, Runnable action)`: Returns a new `CompletionStage` that executes the given `action` after both completes.
- `static CompletableFuture<Void> runAsync(Runnable runnable)`: Returns a new `CompletableFuture` that is asynchronously completed by a task running in the `ForkJoinPool.commonPool()` after it runs the given `action`.

17. [**MRQ**] *(3 points)* Consider the following code fragment:

```
CompletableFuture<Void> a = CompletableFuture.runAsync(() -> {
  sleep();
  System.out.print("A");
});
CompletableFuture<Void> b = a.thenRun(() -> {
  sleep();
  System.out.print("B");
});
CompletableFuture<Void> c = b.thenRun(() -> {
  sleep();
  System.out.print("C");
});
CompletableFuture<Void> d = c.thenRun(() -> {
  sleep();
  System.out.print("D");
});
```

Recap that `System.out.print` does not print a newline character. Select ALL the *possible* print output assuming that `sleep()` may put the thread to sleep for an arbitrary amount of time.

A. `ABCD`
B. `ABDC`
C. `ACBD`
D. `ACDB`
E. `ADBC`
F. `ADCB`

18. [**MRQ**] *(3 points)* Consider the following code fragment:

```
CompletableFuture<Void> a = CompletableFuture.runAsync(() -> {
  sleep();
  System.out.print("A");
});
CompletableFuture<Void> b = a.thenRun(() -> {
  sleep();
  System.out.print("B");
});
CompletableFuture<Void> c = a.thenRun(() -> {
  sleep();
  System.out.print("C");
});
CompletableFuture<Void> d = a.thenRun(() -> {
  sleep();
  System.out.print("D");
});
```

Recap that `System.out.print` does not print a newline character. Select ALL the possible print output assuming that `sleep()` may put the thread to sleep for an arbitrary amount of time.

A. `ABCD`
B. `ABDC`
C. `ACBD`
D. `ACDB`
E. `ADBC`
F. `ADCB`

19. [**MRQ**] *(3 points)* Consider the following code fragment:

```
CompletableFuture<Void> a = CompletableFuture.runAsync(() -> {
  sleep();
  System.out.print("A");
});
CompletableFuture<Void> b = a.thenRun(() -> {
  sleep();
  System.out.print("B");
});
CompletableFuture<Void> c = a.thenRun(() -> {
  sleep();
  System.out.print("C");
});
CompletableFuture<Void> d = c.runAfterBoth(b, () -> {
  sleep();
  System.out.print("D");
});
```

Recap that `System.out.print` does not print a newline character. Select ALL the possible print output assuming that `sleep()` may put the thread to sleep for an arbitrary amount of time.

A. `ABCD`
B. `ABDC`
C. `ACBD`
D. `ACDB`
E. `ADBC`
F. `ADCB`

Section VI: Functor/Monad (12 points)

We will examine the following class:

```
class Counter<T> {
  private final T val;
  private final int count;
  private Counter(T val, int count) {
    this.val = val;
    this.count = count;
  }
  public static <T> Counter<T> of(T val) {
    return new Counter<>(val, 1);
  }
  public <R> Counter<R> map(Immutator<R,T> fn) {
    return new Counter<>(fn.invoke(this.val), this.count+1);
  }
  public <R> Counter<R> flatMap(Immutator<Counter<R>,T> fn) {
    Counter<R> tmp = fn.invoke(this.val);
    return new Counter<>(tmp.val, tmp.count);
  }
  @Override
  public boolean equals(Object obj) {
    if (this == obj) { return true; }
    if (!(obj instanceof Counter<?>)) { return false; }
    Counter<?> ctx = (Counter<?>) obj;
    return this.val.equals(ctx.val) && this.count == ctx.count;
  }
}
```

You are guaranteed that the `Immutator` will not return `null` values or throws an `Exception`. Additionally, we say that two `Counter<T>` is *equal* if and only if both the value `T` are equal based on their `equals` method and the counter `count` stores the same value. Lastly, you do not have to take into account the wild cards.

Select True if the statement below are correct. Otherwise, select False.

20. [**TRUE/FALSE**] *(2 points)* `Counter<T>` satisfies the *left identity law of Monad*?

    Left identity law of Monad: *for any* `x` *and* `f`
    `Counter.of(x).flatMap(y -> f(y))` *is equal to* `f(x)`

21. [**TRUE/FALSE**] *(2 points)* `Counter<T>` satisfies the *right identity law of Monad*?

    Right identity law of Monad: *for any* `monad`
    `monad.flatMap(y -> Counter.of(y))` *is equal to* `monad`

22. [**TRUE/FALSE**] *(2 points)* `Counter<T>` satisfies the *associative law of Monad*?

    Associative law of Monad: *for any* `monad`, `f`, and `g`
    `monad.flatMap(x -> f(x)).flatMap(y -> g(y))`
    is equal to
    `monad.flatMap(x -> f(x).flatMap(y -> g(y)))`

23. [**TRUE/FALSE**] *(2 points)* `Counter<T>` satisfies the *identity morphism preservation of Functor*?

    Identity morphism of Functor: *for any* `functor`
    `functor.map(x -> x)` *is equal to* `functor`

24. [**TRUE/FALSE**] *(2 points)* `Counter<T>` satisfies the *composition morphism preservation of Functor*.

    Composition morphism of Functor: *for any* `functor`, `f`, and `g`
    `functor.map(x -> f(x)).map(y -> g(y))`
    is equal to
    `functor.map(x -> g(f(x)))`

25. [**TRUE/FALSE**] *(1 point)* `Counter<T>` is a **monad**.

26. [**TRUE/FALSE**] *(1 point)* `Counter<T>` is a **functor**.

Section VII: InfiniteList (9 points)

We will consider the following modification to our `InfiniteList`. Instead of a single type `T` (*i.e.,* `InfiniteList<T>`), we want to create an *alternating* infinite list that captures two types. Let us call this `AlternatingList<T,U>`.

You are given the following partial code to create the `AlternatingList<T,U>` as shown below. This is the modification and simplification of `InfiniteList` from the lecture. In particular, note that we cannot call `filter` on this `AlternatingList` and so we are guaranteed that there will be no missing element. You are to make the type *as flexible as possible*.

```
class AlternatingList<T,U> {
  private final Constant<T> head;
  private final Constant</* (A) */> tail;

  private AlternatingList(Constant<T> head, Constant</* (A) */> tail) {
    this.head = head;
    this.tail = tail;
  }

  public T head() {
    return this.head.init();
  }
  public /* (A) */ tail() {
    return this.tail.init();
  }

  public static <T,U> AlternatingList<T,U> generate(
      Constant<? extends T> tSeed,
      Constant<? extends U> uSeed
    ) {
    return new AlternatingList<T,U>(
      /* (B) */,
      /* (C) */
    );
  }
}
```

27. [**MCQ**] *(3 points)* Which of the following type is the *most appropriate* type to replace all occurrences of `/* (A) */`? If there are more than one possible choice, simply choose what you feel is the *most appropriate*.

    A. `AlternatingList<U, T>`

    B. `AlternatingList<U, U>`

    C. `AlternatingList<T, U>`

    D. `AlternatingList<T, T>`

    E. *None of the other*

28. [**MCQ**] *(3 points)* Which of the following expression is the *most appropriate* expression to be used in place of `/* (B) */`? If there are more than one possible choice, simply choose what you feel is the *most appropriate*.

    A. `tSeed`

    B. `uSeed`

    C. `() -> tSeed.init()`

    D. `() -> uSeed.init()`

    E. *None of the other*

29. [**MCQ**] *(3 points)* Which of the following expression is the *most appropriate* expression to be used in place of `/* (C) */`? If there are more than one possible choice, simply choose what you feel is the *most appropriate*.

    A. `generate(tSeed, uSeed)`

    B. `generate(uSeed, tSeed)`

    C. `() -> generate(tSeed, uSeed)`

    D. `() -> generate(uSeed, tSeed)`

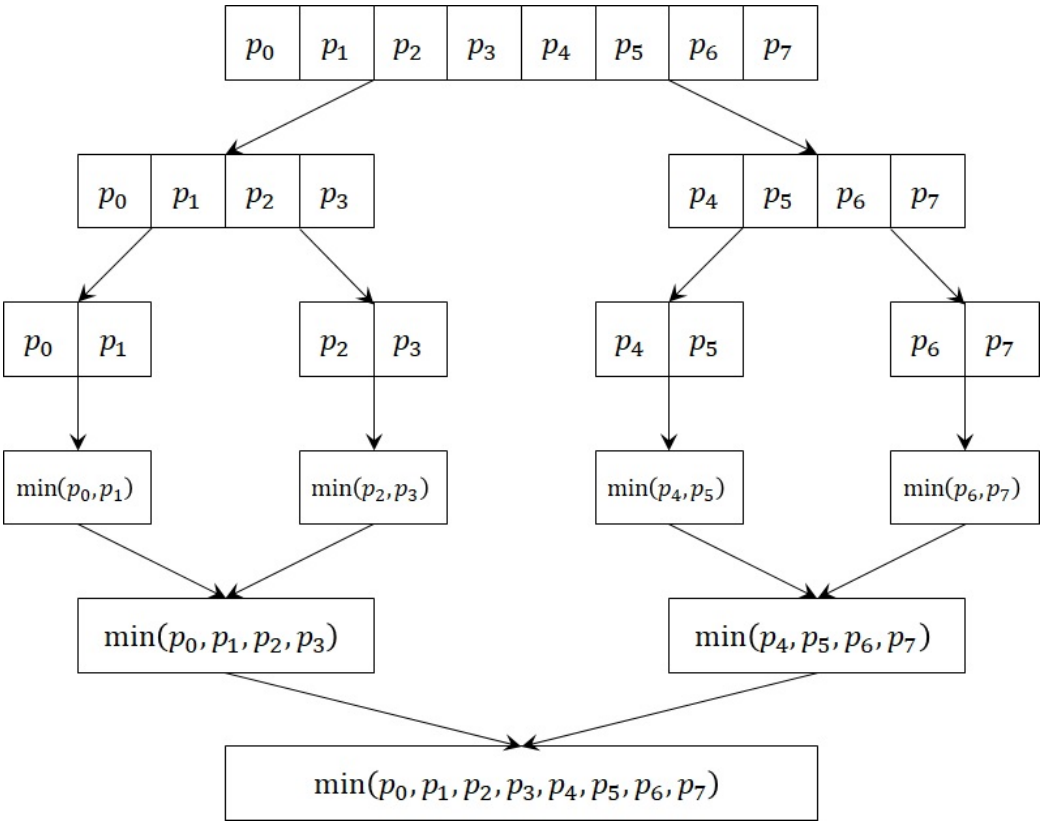
    E. *None of the other*

Section VIII: Parallel Computation (10 points)

We will also be using `RecursiveTask<V>` for this section.

- `protected abstract V compute()`: The main computation performed by this task.
- `public final ForkJoinTask<V> fork()`: Arranges to asynchronously execute this task in the pool the current task is running in, if applicable, or using the `ForkJoinPool.commonPool()` if not `inForkJoinPool()`.
- `final V join()`: Returns the result of the computation when it is done.

The following diagram is a divide and conquer algorithm to find the point in an array of point `arr` that is the nearest to the point `p`.



30. [**ESSAY**] *(10 points)* Write a parallel algorithm to compute the nearest point using `RecursiveTask`. You are given the following template for the class. Your task is to simply override the `compute` method.

    Marks may be deducted if your code is less efficient than what we have discussed in lecture. Your answer should only contain the `compute` method. Any other method will be ignored and may result in deduction of point. Uncompilable code can only receive up to 50% of the mark. You may assume that the array will not be empty.

```
class Point {
  private final double x;
  private final double y;
  public Point(double x, double y) {
    this.x = x; this.y = y;
  }
  public boolean distance(Point p) {
    double dx = this.x - p.x;
    double dy = this.y - p.y;
    return dx*dx + dy*dy;
  }
}
class Closest extends RecursiveTask<Point> {
  private final Point p;
  private final Point[] arr;
  private final int s;
  private final int e;

  // Closest point to p in arr
  // between index s (inclusive) and e (inclusive)
  public Closest(Point p, Point[] arr, int s, int e) {
    this.p = p;
    this.arr = arr;
    this.s = s;
    this.e = e;
  }
}
```

END OF PAPER